Relatório Definitivo de Viabilidade Técnica: Arquitetura e Implementação do Projeto "Circadia" no Zepp OS

A presente avaliação técnica e arquitetónica analisa em exaustivo detalhe a viabilidade de desenvolvimento da aplicação "Circadia", concebida para o ecossistema Zepp OS e direcionada à categoria "Health & Daily Wellness" do Zepp OS Hackathon 2026. O projeto propõe um paradigma avançado de gestão do ciclo circadiano, englobando funcionalidades de *Smart Wake* (despertador inteligente condicionado às fases do sono), deteção heurística de *power naps* (sonecas regenerativas) e consolidação de dados biométricos históricos. A infraestrutura de software delineada assenta numa tripla arquitetura composta por um *Device App* residente no smartwatch, um *Background Service* (AppService) para monitorização contínua e um *Side Service* executado no telemóvel hospedeiro via Zepp App.

A análise subsequente desdobra-se nos seis pilares de viabilidade solicitados, fundamentando cada deliberação técnica estritamente na documentação oficial do Zepp OS, nas limitações impostas pelo sistema operativo de tempo real (RTOS) e nos padrões de comunicação Bluetooth Low Energy (BLE). Todas as restrições sistémicas são dissecadas, sendo propostas soluções de contorno (*workarounds*) arquitetónicas sempre que os bloqueios de firmware inviabilizem a premissa original.

1. Smart Wake (Monitoramento de Sono e Transições em Tempo Real)

A fundação do módulo *Smart Wake* pressupõe a capacidade do smartwatch identificar o exato momento em que o utilizador transita de um estágio de sono profundo para um estágio de sono leve, acionando de imediato o motor háptico (vibração) de forma autónoma, sem requerer interação prévia com o ecrã.

A. Módulos e APIs Utilizados

O desenvolvimento deste módulo requer a orquestração de APIs específicas vocacionadas para a execução em plano de fundo e controlo de hardware:

- **@zos/app-service**: O construtor fundamental para registar e gerir a lógica de negócio na ausência de uma interface de utilizador (UI). Este módulo suporta a política de *Continuous Running*, permitindo a sobrevivência do código após o encerramento do *Device App*[1].
- **@zos/sensor (Módulo HeartRate)**: A interface de leitura da frequência cardíaca, que atua como o principal biomarcador de baixo consumo energético disponível em segundo plano[1].
- **@zos/sensor (Módulo Sleep)**: A abstração nativa do firmware para os algoritmos de classificação de sono, expondo métodos cruciais como getInfo(), getStage() e getSleepingStatus()[2].

- **@zos/sensor (Módulo Vibrator)**: A API responsável por invocar os padrões do motor háptico do relógio através dos métodos start(), stop() e setMode()[3].
- **@zos/sensor (Módulo Time)**: Necessário para agendar subscrições de eventos cadenciados, utilizando o método onPerMinute() para evitar ciclos de processamento contínuos que degradariam a bateria[1].

B. Restrições de Sistema Operacional e Permissões

Para instanciar o serviço em plano de fundo, o ficheiro de configuração app.json deve declarar categoricamente a permissão device:os.bg_service no vetor de permissões e registar o caminho do ficheiro sob a chave module.app-service.services[1]. Adicionalmente, a leitura de dados biométricos exige a permissão data:user.hd.sleep para o sensor de sono[2].

A restrição mais crítica imposta pelo Zepp OS reside na classificação de consumo energético dos sensores. O firmware do Zepp OS, operando sob princípios estritos de um RTOS para maximização da bateria, desativa sumariamente o acesso a sensores de elevado consumo computacional e energético quando o contexto de execução não possui uma UI ativa. Consequentemente, **interfaces de alto consumo como Accelerometer, Geolocation e Gyroscope estão liminarmente bloqueadas no escopo do AppService**[1]. A tentativa de aceder aos dados vetoriais em bruto do acelerómetro em tempo real a partir do serviço em background resultará em falha silenciosa ou no encerramento forçado do processo pelo *watchdog* do sistema operativo.

Em contrapartida, sensores classificados como de baixo consumo energético, especificamente o HeartRate, são explicitamente suportados no ambiente do AppService[1]. No que concerne ao Vibrator, a documentação indica que a única forma de interagir visualmente com o utilizador a partir de um serviço é através da API @zos/notification[1]. Não obstante, a evocação autónoma e programática do hardware de vibração via método Vibrator.start() não sofre bloqueios de interface, permitindo que o relógio vibre sem necessariamente apresentar uma notificação gráfica imediata[3].

C. Veredito de Viabilidade Técnica: Parcialmente Possível

A premissa exata idealizada — interrogação constante do acelerómetro e do sensor de ritmo cardíaco em bruto para cálculo heurístico próprio durante a noite num *AppService* — é **Impossível** devido aos limites impostos pelo kernel ao acesso do acelerómetro em segundo plano[1]. Contudo, a entrega da funcionalidade *Smart Wake* em si é viável através de uma solução arquitetónica que confie no processamento nativo do relógio.

O *workaround* ideal consiste em delegar a complexidade do cálculo cinético ao coprocessador do Zepp OS e interrogar a API estática de Sleep. O sensor de sono avalia continuamente as fases e expõe os resultados através do método getStage(), que devolve um vetor contendo os estágios classificados de acordo com as constantes nativas apresentadas na tabela abaixo:

Constante de Estágio	Tipo Numérico	Descrição Fisiológica	API_LEVEL
WAKE_STAGE	number	Estágio de vigília	2.0

		(acordado).	
REM_STAGE	number	Estágio de movimento rápido dos olhos (REM).	2.0
LIGHT_STAGE	number	Estágio de sono leve.	2.0
DEEP_STAGE	number	Estágio de sono profundo.	2.0

O código deve instanciar um subscritor de tempo que, na janela de despertar definida pelo utilizador (por exemplo, entre as 06:30 e as 07:00), verifica minuto a minuto se o último elemento do vetor getStage() corresponde ao LIGHT_STAGE. Caso a condição se verifique, o alarme é disparado de forma autónoma utilizando um padrão de vibração de alta prioridade.

```JavaScript
import { AppService } from '@zos/app-service'
import { Sleep, Vibrator, Time } from '@zos/sensor'
import { VIBRATOR_SCENE_TIMER } from '@zos/sensor'

const timeSensor = new Time()
const sleepSensor = new Sleep()
const vibrator = new Vibrator()

// Janela de despertar configurada (Ex: 06:30 às 07:00)
const wakeWindowStart = 6 * 60 + 30; // 390 minutos
const wakeWindowEnd = 7 * 60;        // 420 minutos

AppService({
  onInit(options) {
    timeSensor.onPerMinute(() => {
      const currentMinutes = timeSensor.getHours() * 60 + timeSensor.getMinutes();

      // Verifica se está dentro da janela de Smart Wake
      if (currentMinutes >= wakeWindowStart && currentMinutes <= wakeWindowEnd) {
```

```
        const stages = sleepSensor.getStage()
        const sleepConstants = sleepSensor.getStageConstantObj()

        if (stages && stages.length > 0) {
            const currentStage = stages[stages.length - 1].model

            // Dispara se transitar para sono leve ou se a janela estiver no fim
            if (currentStage === sleepConstants.LIGHT_STAGE || currentMinutes ===
wakeWindowEnd) {
                vibrator.setMode(VIBRATOR_SCENE_TIMER);
                vibrator.start()
                // Aqui também seria disparado o comando ZML para o Side Service
            }
        }
    })
}
```

Esta abordagem contorna o bloqueio do acelerómetro consumindo apenas ciclos mínimos de CPU a cada minuto, garantindo total conformidade com as exigências do Zepp OS[1].

2. Side Service (Comunicação e Alerta Sonoro no Celular)

O *Side Service* tem a responsabilidade de estabelecer uma ponte de comunicação com o smartphone, recebendo o evento de despertar emitido pelo smartwatch e acionando um alerta sonoro. A criticidade deste requisito repousa na capacidade técnica de o telemóvel soar um alarme audível mesmo que o sistema operativo do smartphone (iOS ou Android) se encontre em modo "Não Incomodar" (Do Not Disturb - DND) ou com o ecrã bloqueado.

A. Módulos e APIs Utilizados

O ecossistema Zepp abstrai a comunicação Bluetooth através de uma camada de mensagens dedicada.

- **Módulo Messaging API**: Canal de comunicação primário base, suportando exclusivamente transferência de dados binários (Buffer/ArrayBuffer) entre o *Device App* e o *Side Service*[5].
- **Bibliotecas /shared/message.js e /shared/message-side.js**: Revestimentos (*wrappers*) arquitetónicos oficiais baseados no padrão MessageBuilder, desenhados para facilitar o envio e a receção de objetos JSON serializados sobre a Messaging API, utilizando uma semântica orientada a eventos (call, request, response)[6].
- **Fetch API (network.downloader)**: Interface disponível exclusivamente no *Side Service* que permite a execução de pedidos HTTP/HTTPS para serviços web externos[7].

B. Restrições de Sistema Operacional e Permissões

A comunicação entre o AppService e o *Side Service* via protocolo BLE é plenamente suportada e projetada para baixa latência. Exceptuando as APIs que se iniciam com o prefixo mst (como certos envios multimédia), o módulo @zos/ble opera livremente em serviços de plano de fundo[1].

O constrangimento arquitetónico fatal reside na natureza do ambiente de execução do *Side Service*. O *Side Service* não se consubstancia numa aplicação nativa tradicional codificada em Swift (iOS) ou Kotlin (Android). Em vez disso, consiste numa área de segurança isolada (*JavaScript sandbox*) encapsulada, que é interpretada em tempo de execução pelos motores JavaScript embutidos na aplicação principal Zepp App[5].

Devido a esta contentorização rigorosa, a sandbox JS não expõe pontes nativas (OS bridges) para manipular APIs de baixo nível do smartphone. Não existem métodos disponíveis no catálogo do *Side Service API* capazes de interagir com as sessões de áudio do sistema (AVAudioSession no iOS ou AudioManager no Android) para solicitar foco de áudio crítico, forçar o encaminhamento de som pelo altifalante principal contornando os perfis de interrupção silenciados, ou acordar o ecrã do telemóvel por cima da fechadura de segurança[5]. A execução cinge-se essencialmente à transferência de dados, gestão de definições internas da aplicação (settingsStorage) e chamadas de rede genéricas[5].

C. Veredito de Viabilidade Técnica: Parcialmente Possível

É perfeitamente **Possível** transmitir o comando instantâneo do relógio para o telemóvel através de ZML/Messaging em milissegundos assim que a fase de sono ideal for detetada[5]. Contudo, é estritamente **Impossível** que o próprio script do *Side Service* force, de forma independente, a reprodução de um alarme sonoro que ultrapasse os modos silenciosos ou de bloqueio do smartphone.

Como estratégia de mitigação, a conceção do "Circadia" deverá alavancar a Fetch API presente no *Side Service* para orquestrar o evento através de um vetor externo com permissões nativas ou de rede doméstica[7].

O fluxo de dados da solução de contorno envolveria os seguintes passos:

1. O *AppService* identifica a transição para sono leve e dispara um comando via MessageBuilder.call({ action: 'WAKE_UP' }) para o telemóvel[6].
2. O *Side Service* capta o evento na sua rotina onInit() e emite imediatamente um pedido HTTP POST (webhook) não bloqueante para plataformas de automação (tais como IFTTT, Tasker via AutoRemote, ou atalhos da Apple configurados para receção web).
3. A aplicação de automação instalada no smartphone, esta sim detentora de permissões ao nível do OS, interpreta o webhook e executa a rotina nativa para desativar o modo DND, maximizar o volume e reproduzir a sirene de alarme, ou mesmo acender o sistema de iluminação inteligente do quarto.

```javascript
// Exemplo de execução no Side Service (app-side/index.js)
import { MessageBuilder } from '../shared/message-side'

const messageBuilder = new MessageBuilder()

AppSideService({
  onInit() {
    messageBuilder.listen(() => {})

    // Ouve o evento vindo do relógio
    messageBuilder.on('call', (req) => {
      const payload = messageBuilder.buf2Json(req.payload)

      if (payload.action === 'WAKE_UP') {
        // Aciona webhook externo para contornar limitações nativas de áudio
        fetch({
          url: 'https://maker.ifttt.com/trigger/smart_wake/with/key/YOUR_IFTTT_KEY',
          method: 'POST'
        }).then(response => {
          console.log('Webhook de despertar disparado com sucesso')
        }).catch(error => {
          console.error('Falha ao comunicar com o serviço de automação')
        })
      }
    })
  }
})
```

3. True-Sleep Power Nap (Estimativa de Início do Sono via Sensores)

A funcionalidade *True-Sleep Power Nap* objetiva o cálculo de uma heurística matemática customizada no relógio para determinar com exatidão forense o momento em que o utilizador adormeceu. Para tal, a proposta visa combinar a análise da diminuição aguda da frequência cardíaca com a cessação de micro-movimentos detetados pelo acelerómetro, de modo a fixar o *timestamp* exato e subtrair o período de latência de sono da duração da soneca.

A. Módulos e APIs Utilizados

- **@zos/sensor (Módulo Accelerometer)**: API que expõe os vetores de força gravitacional (X, Y, Z) e métricas de repouso através de retornos de chamada (callbacks) gerados por onChange() e métodos de consulta direta como getCurrent()[9]. Permite a afinação da resolução temporal através da propriedade setFreqMode().

- **@zos/sensor (Módulo HeartRate)**: Leitura da pulsação instantânea contínua[10].
- **@zos/sensor (Módulo Sleep)**: API consolidada de sono que comporta propriedades nativas direcionadas às métricas de sestas diurnas[2].

B. Restrições de Sistema Operacional e Permissões

A documentação sobre o Accelerometer é detalhada no que respeita aos seus modos de operação. Através do método setFreqMode(), os programadores podem induzir o sensor a operar em três cadências distintas: FREQ_MODE_LOW (baixa frequência de disparo, vocacionado para poupança de energia), FREQ_MODE_NORMAL (frequência média) e FREQ_MODE_HIGH (alta frequência para desportos de alta intensidade e cálculos cinéticos refinados)[9].

Todavia, colidimos novamente com as fronteiras rígidas do ambiente de execução em plano de fundo abordadas no Ponto 1. A arquitetura RTOS do Zepp inviabiliza que *qualquer* sensor de alto consumo, independentemente da granularidade requisitada, seja ativado pelo AppService[1]. A leitura paralela dos eixos espaciais exige que a CPU seja impedida de transitar para os estados profundos de suspensão (deep sleep states). Caso o programador tentasse contornar a regra instanciando a leitura vetorial dentro do ecrã principal (mantendo o *Device App* em primeiro plano e, hipoteticamente, forçando o ecrã a manter-se aceso ininterruptamente, embora isso seja fortemente desencorajado por riscos evidentes de envelhecimento do painel AMOLED e exaustão rápida da célula de lítio), os dados seriam obtidos, mas ao custo de arruinar a vida útil da bateria para uma simples soneca de trinta minutos[1].

C. Veredito de Viabilidade Técnica: Parcialmente Possível

O desenvolvimento de um algoritmo matemático customizado baseado em dados cinéticos *raw* extraídos em segundo plano é categoricamente **Impossível**.

O contorno arquitetónico apropriado não consiste em subverter o acelerómetro, mas sim em confiar nos sofisticados motores de aprendizagem computacional incorporados no firmware de biometria da Amazfit. A classe Sleep dispõe de uma abstração perfeitamente alinhada com as exigências da aplicação: o método getNap()[2]. Este método, introduzido de forma estável no Zepp OS 3.0, efetua exatamente o cálculo heurístico (incluindo processamento vetorial de inércia a nível de C++) que a aplicação tentaria replicar, libertando o ambiente JavaScript desse peso computacional.

O retorno de getNap() é um vetor (array) populado por objetos cujas propriedades descrevem minuciosamente os episódios de sono diurno[2].

Propriedade	**Tipo Numérico**	**Descrição Fisiológica do NapInfo**	**API_LEVEL**
length	number	Duração total consolidada da soneca em minutos, expurgada da	3.0

		latência de início.	
start	number	*Timestamp* de início da soneca, expresso em minutos passados após as 0:00h do dia corrente.	3.0
stop	number	*Timestamp* de término da soneca, expresso em minutos passados após as 0:00h.	3.0

Para uma precisão em tempo real sem esperar pela consolidação do getNap(), a aplicação deve observar alternativamente a mudança de estado através do método getSleepingStatus(), que comuta de 0 (vigiado) para 1 (dormindo), ou aplicar matemáticas secundárias exclusivamente sobre as oscilações da métrica HeartRate, que é inerentemente permissível no AppService[1].

4. Circadian Dashboard (Acesso a Dados Históricos e Biométricos)

A eficácia do *Circadian Dashboard* depende da leitura, agregação e armazenamento permanente do rasto fisiológico do utilizador transposto de dias e noites precedentes. O objetivo fulcral é a computação de estimativas relativas à "pressão de sono" (necessidade fisiológica de repouso induzida pelo acúmulo de adenosina e pelo ritmo circadiano). Para isso, a arquitetura demanda acesso a dados do sono noturno, à Variabilidade da Frequência Cardíaca (HRV) e ao desgaste inerente à atividade física diária.

A. Módulos e APIs Utilizados

- **@zos/sensor (Módulo Sleep)**: API provedora da totalidade da arquitetura do sono noturno. Os dados retroativos são acedidos via getInfo(), que retorna um objeto SleepInfo[2].
- **@zos/sensor (Módulo Workout)**: Fornece o acervo histórico de esforços cardiovasculares, devolvendo as cargas de treino via getHistory() e os patamares basais das zonas de frequência cardíaca via getUserHrZoneSettings()[11].
- **@zos/sensor (Módulo Stress e checkSensor)**: Interfaces fundamentais para capturar o desgaste do sistema nervoso autónomo[12].
- **@zos/storage (LocalStorage)**: Mecanismo primário de persistência para armazenar as

estimativas calculadas, sobrevivendo aos ciclos de vida do *Device App*[14].

B. Restrições de Sistema Operacional e Permissões

O acesso a este mosaico de informações de saúde invoca a necessidade imperativa de aprovações de acesso a dados privados. Os códigos data:user.hd.sleep e data:user.hd.workout têm de figurar entre as permissions do ficheiro manifest[2].

A análise da documentação sobre o cálculo do HRV revela uma assimetria técnica no ecossistema atual. Embora o Zepp OS apresente suporte robusto e granular a métricas contínuas, não há evidência de que os dados cruzados de *Heart Rate Variability* em domínio de tempo (ex: SDNN, RMSSD) estejam expostos ao nível da programação de aplicações em JavaScript sob a forma de uma classe bruta @zos/sensor instanciável por terceiros para processamento arbitrário. O HRV e outras variáveis especializadas encontram-se predominantemente limitadas a identificadores de visualização direta e passiva em componentes gráficos de *Watchfaces* através de constantes como hmUI.data_type.HRV, hmUI.data_type.TRAINING_LOAD e hmUI.data_type.SLEEP[17]. Isto implica que, para a aplicação desenhar o seu dashboard, não é possível iterar pelas flutuações de batimento-a-batimento.

No campo do armazenamento e retenção histórica, o Zepp OS oferece métodos adequados, divididos categoricamente consoante a volatilidade necessária[15]. O armazenamento intermédio e transversal é suportado pelas APIs na namespace @zos/storage, superando largamente a morosidade imputada às operações diretas do @zos/fs (File System)[16].

API de Armazenamento	**Módulo Referência**	**Persistência e Ciclo de Vida**	**Contexto de Utilização Recomendado**	**API_LEVEL**
SessionStorage	@zos/storage	Temporária. Limpa-se ao descarregar a aplicação.	Transferência isolada de dados entre páginas numa única sessão.	2.0
LocalStorage	@zos/storage	Permanente. Destrói-se apenas com a desinstalação.	Histórico de pressão de sono, preferências, base de dados diária (JSON stringified).	3.0
FileSystem	@zos/fs	Permanente. Baseado em	Escrita de ficheiros	1.0 / 2.0

		inodes do RTOS.	binários complexos ou grandes volumes de log.	

O LocalStorage detém a eficiência premente pois efetua o carregamento antecipado (*lazy loading*) da base de chaves-valor para a memória RAM (a memória estática), reduzindo a fricção e o consumo de energia associados a sucessivas chamadas de leitura/escrita a sistemas de ficheiros clássicos[15]. O seu uso demanda a permissão device:os.local_storage[15].

C. Veredito de Viabilidade Técnica: Possível

O projeto arquitetural do *Circadian Dashboard* é totalmente **Possível**, com a ressalva da ausência de dados nativos granulares de HRV, que devem ser suplementados pela aproximação utilizando as APIs de Stress e Workout como métricas proxy do esforço do utilizador.

A mecânica subjacente para cimentar as inferências históricas pode ser programada numa rotina de consolidação executada diariamente, de preferência no momento em que a fase WAKE_STAGE for consolidada. Os atributos do sono, fornecidos sob a nomenclatura da interface genérica SleepInfo, contêm as frações temporais exatas que norteiam os modelos fisiológicos:

```
JavaScript
import { Sleep, Workout } from '@zos/sensor'
import { LocalStorage } from '@zos/storage'

const sleep = new Sleep()
const workout = new Workout()
const localStorage = new LocalStorage()

export function consolidateCircadianMetrics() {
  // Obtém o objeto SleepInfo com score, deepTime e totalTime (minutos)
  const sleepInfo = sleep.getInfo()

  // Exemplo: Cálculo simples de déficit de pressão do sono
  // Assumindo a necessidade ótima de 480 minutos (8 horas)
  const baseRequirement = 480
  let sleepDeficit = baseRequirement - sleepInfo.totalTime
```

```
  if (sleepDeficit < 0) sleepDeficit = 0

  const circadianModel = {
    timestamp: new Date().getTime(),
    deficitMinutes: sleepDeficit,
    deepSleepRatio: sleepInfo.deepTime / sleepInfo.totalTime,
    nightScore: sleepInfo.score
  }

  // Armazenamento estruturado e persistente do modelo em formato JSON
  const existingHistory = JSON.parse(localStorage.getItem('circadian_db') || '[]');
  existingHistory.push(circadianModel);

  // Limita o histórico aos últimos 14 dias para evitar Memory Leaks
  if (existingHistory.length > 14) {
    existingHistory.shift();
  }

  localStorage.setItem('circadian_db', JSON.stringify(existingHistory));
```

A combinação das propriedades do sensor de treino, como os escalões biológicos providenciados por getUserHrZoneSettings()[11], interligada com as proporções extraídas do SleepInfo[2], proporcionará uma profundidade assinalável ao projeto, suplantando largamente as expetativas dos jurados do Hackathon no que diz respeito à sofisticação da análise biométrica em *wearables* RTOS.

5. Execução Contínua e Bateria (Limites do AppService)

A viabilidade global do projeto "Circadia" alicerça-se na resistência da monitorização ao longo de um extenso ciclo ininterrupto. O construto de um serviço perene que avalie, ao longo de oito horas noturnas consecutivas, a fisiologia do utilizador obriga a uma compreensão imaculada das políticas sancionatórias do escalonador (scheduler) do kernel para os enquadramentos de plano de fundo.

A. Módulos e APIs Utilizados

- **@zos/app-service**: O gestor do ciclo de vida, regido pelos comandos diretos @zos/app-service start, @zos/app-service stop e exit()[1].
- **System Events**: Arquitetura de notificação semântica introduzida no Zepp OS v3 que transmite perturbações e flutuações contextuais e fisiológicas ao AppService através da assinatura de tópicos de eventos (ex: event:os.health.sleep_status)[22].
- **@zos/app**: Gestão obrigatória de permissões perante o utilizador final via queryPermission e requestPermission[1].

B. Restrições de Sistema Operacional e Permissões

O Zepp OS subdivide a natureza da lógica sem interface em dois arquétipos estruturais de comportamento:

1. **Single Execution (Execução Única)**: Ativada por eventos episódicos, como alarmes, notificações ou transições do sistema. O serviço desperta, cumpre as suas instruções processuais, e finaliza sumariamente. Este formato goza de elevado privilégio aos olhos do OS, não exigindo a manutenção de *heaps* complexos de memória e, por conseguinte, consumindo quantidades irrisórias de energia térmica e eletromagnética[1].
2. **Continuous Running (Execução Contínua)**: Requisitado pelo método start sob a alçada da permissão device:os.bg_service. Uma vez iniciado, a estrutura do serviço, inclusive após a devolução explícita à UI do dispositivo ou a ocorrência de episódios de reinício forçado (*reboot*), perdura de forma persistente no sistema operativo[1].

As políticas de terminação forçada (*kill policies*) para um contexto prolongado de oito horas impõem desafios intrincados. Devido ao volume ínfimo de memória RAM dedicada e ao microprocessador de baixo clock do smartwatch, a estabilidade global não pode ser ameaçada.
A primeira restrição manifesta-se nos vetores de simultaneidade. O sistema garante inabalavelmente que **apenas uma instância de uma aplicação em execução contínua se encontra ativa a qualquer dado instante**[1]. Na contingência de a "Circadia" solicitar a execução em pano de fundo e deparar-se com uma concorrência por parte de uma aplicação prévia já residente nesse espetro (ex: uma aplicação paralela de mapas georreferenciados), o RTOS não executa um encerramento passivo. Em contrapartida, levanta imperativamente uma caixa de diálogo interativa que confia no discernimento do utilizador final para eleger o processo sobrevivente[1]. O não consentimento através dos pedidos dinâmicos de requestPermission culminará no bloqueio imediato do escalonador para a referida tarefa[1].
A segunda e mais imperdoável barreira concerne ao abuso de ciclos de computação. Instâncias que não respeitem a ociosidade (*idling*) do microcontrolador desencadeiam a sanção máxima por excesso de drenagem, resultando num colapso silenciado do processo. A documentação exige que um serviço contínuo durma a maior parte da sua existência[1].

C. Veredito de Viabilidade Técnica: Possível (Mediante Implementação de Boas Práticas)

É perfeitamente **Possível** orquestrar a atividade noturna ininterrupta sem risco eminente de terminação forçada pelo kernel por motivos de inatividade do ecrã ou consumo energético, desde que a abstração assente rigorosamente no modelo passivo conduzido por eventos.
O padrão arquitetónico inadequado – passível de desfechos fatais, sob a égide da degradação térmica – envolveria a instanciação de bucles infinitos ou de temporizadores JavaScript em alta cadência (setInterval(function, 1000)) para forçar o exame do estágio de sono.
Em contraponto, as boas práticas determinam que a aplicação confie nas interrupções ao nível do sistema operativas providenciadas pelo quadro de *System Events*.
Desde o limiar do Zepp OS v3 (API_LEVEL 3.0), as métricas contextuais podem ser mapeadas na tabela de rotas do app.json submetendo o AppService ao processamento seletivo de *triggers* do kernel, os quais passam propriedades estruturais codificadas via notação URI

(*query string*) ao método iterativo onInit(options)[22].

Evento de Sistema (System Event)	Descrição do Trigger no Firmware	Argumentos Extra Passados	API_LEVEL
event:os.health.sleep_status	Alteração nativa no estado de sono (vigília para repouso).	-	3.0
event:os.system.sleep_mode	Comutação de hardware no modo DND/Sono.	status=enter, status=exit	3.0
event:os.health.heart_rate_abnl	Disparo originado por uma ritmia anómala fora dos intervalos aceitáveis.	-	3.0

Delineia-se, assim, uma arquitetura de sobrevivência garantida: a aplicação subscreve explicitamente o evento de sistema para as transições de sono e permanece em inércia estrutural profunda (sem afetar o escalonador). Adicionalmente, ampara-se no método onPerMinute do sensor Time para assegurar avaliações leves e regulares com janelas de interrupção cronometradas a exatos 60.000 milissegundos[1].

6. Compatibilidade de Dispositivos e API_LEVEL

A corroboração empírica final para o projeto destina-se à aferição da viabilidade das peças de *hardware* visadas, averiguando se as ramificações dos *smartwatches* de destino ostentam a infraestrutura e os interpretadores atualizados para digerir a plenitude do ecossistema proposto — que congrega *AppService*, comunicações em segundo plano via ZML/Messaging e funcionalidades exigidas a nível do API_LEVEL 5.0.

A. Módulos e APIs Utilizados

- A taxonomia de identificação destas caraterísticas incide nos registos oficiais da matriz global de dispositivos da fabricante e na consulta indireta através do método global getDeviceInfo() proveniente de hmSetting (versões arcaicas) e getSystemInfo() para confirmação sistemática de resolução de software e níveis de API ativos[10].

B. Restrições de Sistema Operacional e Permissões

O pacote de inovações do Zepp OS não decorre retroativamente na sua íntegra a arquiteturas obsoletas. A incorporação formal dos mecanismos de *App Service*, comunicações

inter-processos enriquecidas entre o Device App e o *Side Service*, envio de ficheiros bidirecionais e as interrupções biológicas padronizadas pelos System Events datam das fundações monumentais efetuadas com a introdução do Zepp OS v3 (API_LEVEL 3.0)[22]. A progressão contínua estabilizou módulos refinados nas sequências das APIs 3.6, 4.0 e, em última instância, culminou nas premissas que governam a atual vanguarda de desenvolvimento para os programadores inseridos no Hackathon 2026.

C. Veredito de Viabilidade Técnica: Possível

A implantação do ecossistema completo pretendido sobrepõe-se maravilhosamente e é inteiramente **Possível** sob as balizas definidas para o concurso, desde que o parque de equipamentos corresponda rigorosamente às mais recentes estirpes da marca.
A consulta à matriz de dispositivos (Devices with Zepp OS) elucida categoricamente o estalão tecnológico dos alvos premium visados pelo "Circadia", notadamente os pilares da família "Balance"[25]. Note-se o sumário de compatibilidade detalhado abaixo, abarcando as variantes redondas (Round) e retangulares (Square) cujas silhuetas enquadram o espetro visual do projeto:

Equipamento / Firmware Primordial	**Formato do Ecrã**	**API_LEVEL Vigente**	**Versão do Zepp OS**	**Suporte a SecondaryWidget**
Amazfit Balance 3 (Alvo Primário)	Redondo (480x480)	**4.4 / 5.0**	**5.0**	NÃO
Amazfit Balance Ultra	Redondo (480x480)	**4.4 / 5.0**	**5.0**	NÃO
Amazfit Bip Max	Quadrado (432x514)	**4.4 / 5.0**	**5.0**	NÃO
Amazfit Cheetah 2 Ultra	Redondo (480x480)	**4.3 / 5.0**	**5.0**	NÃO
Active 3 Premium	Redondo (466x466)	**4.2 / 5.0**	**5.0**	NÃO
Amazfit T-Rex 3 Pro (48mm)	Redondo (480x480)	**4.2 / 5.0**	**5.0**	NÃO

Amazfit Balance 2 (Anterior)	Redondo (480x480)	4.2	5.0	SIM
Amazfit T-Rex 3 (Anterior)	Redondo (480x480)	4.0	4.5	NÃO
Amazfit Active (Anterior)	Quadrado (390x450)	3.6	4.0	SIM

Uma ressalva interpretativa crucial impera no tocante à nomenclatura de versionamento mencionada no enunciado do projeto. Enquanto as diretrizes apontam "API_LEVEL 5.0 (Zepp OS 6)", as tabulações matriciais vigentes na base de dados arquitetónica para o desenvolvimento presente estipulam que o auge dos equipamentos contemporâneos (séries Balance 3, Balance Ultra e Bip Max) encontra-se firmemente estabelecido no alicerce normativo de compilação listado como "Latest API_LEVEL 4.4 / 5.0" correlacionado indissociavelmente ao "Latest Zepp OS Version 5.0"[25]. Independentemente desta defasagem terminológica, todos os vértices de software que garantem o ciclo virtuoso do sistema (desde a alocação LocalStorage e os eventos de oscilação temporal até aos constructos binários do BLE *MessageBuilder*) encontram-se imutavelmente ativados e nativamente operacionais[6].
Deste modo, o Amazfit Balance responde afirmativamente, não ostentando qualquer restrição, a totalidade das caraterísticas exaustivas solicitadas para a implantação do "Circadia", erguendo-se como o recetáculo natural e por excelência para albergar a engenharia de precisão descrita ao longo deste relatório de viabilidade[25].

Works cited

1. App Service | Zepp OS Developers Documentation, https://docs.zepp.com/docs/guides/framework/device/app-service/
2. Sleep | Zepp OS Developers Documentation, https://docs.zepp.com/docs/reference/device-app-api/newAPI/sensor/Sleep/
3. Vibrator | Zepp OS Developers Documentation, https://docs.zepp.com/docs/reference/device-app-api/newAPI/sensor/Vibrator/
4. Time | Zepp OS Developers Documentation, https://docs.zepp.com/docs/reference/device-app-api/newAPI/sensor/Time/
5. Messaging API | Zepp OS Developers Documentation, https://docs.zepp.com/docs/reference/side-service-api/messaging/
6. MessageBuilder Bluetooth Communication - Zepp OS, https://docs.zepp.com/docs/guides/best-practice/bluetooth-communication/
7. Download File | Zepp OS Developers Documentation, https://docs.zepp.com/docs/reference/side-service-api/download-file/

8. Settings Storage API | Zepp OS Developers Documentation, https://docs.zepp.com/docs/reference/side-service-api/settings-storage/
9. Accelerometer | Zepp OS Developers Documentation, https://docs.zepp.com/docs/reference/device-app-api/newAPI/sensor/Accelerometer/
10. Zepp OS 2.1 New Features, https://docs.zepp.com/docs/guides/version-info/new-features-21/
11. Workout | Zepp OS Developers Documentation, https://docs.zepp.com/docs/reference/device-app-api/newAPI/sensor/Workout/
12. STRESS | Zepp OS Developers Documentation, https://docs.zepp.com/docs/watchface/api/hmSensor/sensorId/STRESS/
13. checkSensor | Zepp OS Developers Documentation, https://docs.zepp.com/docs/reference/device-app-api/newAPI/sensor/checkSensor/
14. LocalStorage | Zepp OS Developers Documentation, https://docs.zepp.com/docs/reference/device-app-api/newAPI/share-storage/LocalStorage/
15. LocalStorage | Zepp OS Developers Documentation, https://docs.zepp.com/docs/reference/device-app-api/newAPI/storage/localStorage/
16. Data Persistence | Zepp OS Developers Documentation, https://docs.zepp.com/docs/guides/best-practice/persistence-storage/
17. GRADKIENT_POLYLINE | Zepp OS Developers Documentation, https://docs.zepp.com/docs/watchface/api/hmUI/widget/GRADKIENT_POLYLINE/
18. Release history | Zepp OS Developers Documentation, https://docs.zepp.com/docs/guides/tools/watchface/release-note/
19. data_type | Zepp OS Developers Documentation, https://docs.zepp.com/docs/watchface/api/hmUI/widget/data_type/
20. SessionStorage | Zepp OS Developers Documentation, https://docs.zepp.com/docs/reference/device-app-api/newAPI/storage/sessionStorage/
21. Migration from version 1.0 | Zepp OS Developers Documentation, https://docs.zepp.com/docs/v2/guides/version-info/migration-guide/
22. System Events | Zepp OS Developers Documentation, https://docs.zepp.com/docs/guides/framework/device/system-event/
23. Zepp OS 3.0 New Features, https://docs.zepp.com/docs/guides/version-info/new-features-30/
24. Revision History | Zepp OS Developers Documentation, https://docs.zepp.com/docs/1.0/reference/revision-history/
25. Device Basic Information | Zepp OS Developers Documentation, https://docs.zepp.com/docs/reference/related-resources/device-list/